ЛАБОРАТОРНАЯ РАБОТА №5

Модуль 2. Часть 2. Базовые типы данных – строки. Методы строк

Цель работы: формирование опыта работы с базовым типом – строки, изучить методы строк.

Методы строк

Методы — это функции, которые ассоциированы с определенными объектами и действуют на них. Формально они являются атрибутами, прикрепленными к объектам, которые ссылаются на вызываемые функции, всегда имеющие в своем распоряжении подразумеваемый объект.

Говоря точнее, функции представляют собой пакеты кода, а вызовы методов комбинируют две операции — извлечение атрибута и вызов.

Извлечения атрибутов

Выражение в форме объект, атрибут означает “извлечь значение атрибута в объекте”.

Выражения вызовов

Выражение в форме функция (аргументы) означает “вызвать код функции, передав ему ноль или большее количество разделенных запятыми объектов аргументов, и возвратить результирующее значение функции".

Объединение двух операций вместе позволяет вызвать метод объекта. Выражение вызова метода:

объект. метод (аргументы)

оценивается слева направо — Python сначала извлечет метод из объекта и затем вызовет его, передавая объект и аргументы.

```
>>> s = ’spam’
>>> result = s.find(’pa’)          # Вызов метода find для поиска
                                   # 'pa' в строке S
```

В общем случае для создания нового значения из существующей строки мы конструируем новую строку с помощью операций вроде нарезания и конкатенации.

Скажем, для замены двух символов в середине строки можно применять такой код:

```
>>> S = 'spammy'
>>> S = S[:3] + ’xx' + S[5:]     # Нарезать сегменты из S
>>> S
```

```
'spaxxy'
```

Но если нужно только заменить подстроку, тогда можно использовать метод **replace** (замена):

```
>>> S = 'spammy'
>>> S = S. replace (' mm ' , 'xx')       # Заменить все mm на xx в S
>>> S
'spaxxy'
```

Метод replace является более универсальным, чем вытекает из приведенного кода. Он принимает в качестве аргументов исходную подстроку (любой длины) и строку (любой длины), которой должна быть заменена исходная подстрока, и выполняет глобальный поиск и замену:

```
>>>'aa$bb$cc$dd'.replace('$', 'SPAM')
'aaSPAMbbSPAMccSPAMdd'
```

В такой роли метод replace может применяться как инструмент для реализации замен по образцу (например, в форматированных бланках).

Основные методы строк (список)

```
S.capitalize()                          S.ljust(width [, fill])
S.casefold()                            S.lower()
S.center(width [, fill])                S.lstrip([chars])
S.count(sub [, start [, end]])          S.maketrans(x[, y[, z]])
S.encode([encoding [,errors]])          S.partition(sep)
S.endswith(suffix [, start [, end]])    S.replace(old, new [, count])
S.expandtabs([tabsize])                 S.rfind(sub [,start [,end]])
S.find(sub [, start [, end]])           S.rindex(sub [, start [, end]])
S.format(fmtstr, *args, **kwargs)       S.rjust(width [, fill])
S.index(sub [, start [, end]])          S.rpartition(sep)
S.isalnum()                             S.rsplit([sep[, maxsplit]])
S.isalpha()                             S.rstrip([chars])
S.isdecimal()                           S.split([sep [,maxsplit]])
S.isdigit()                             S.splitlines([keepends])
S.isidentifier()                        S.startswith(prefix [, start [, end]])
S.islower()                             S.strip([chars])
S.isnumeric()                           S.swapcase()
S.isprintable()                         S.title()
S.isspace()                             S.translate(map)
S.istitle()                             S.upper()
S.isupper()                             S.zfill(width)
S.join(iterable)
```

Если требуется заменить одну подстроку фиксированного размера, которая может располагаться по любому смещению, то можно сделать замену снова или выполнить поиск подстроки посредством метода find и затем произвести срез:

```
>>> S = 'xxxxSPAMxxxxSPAMxxxx'
>>> where = S.find('SPAM')          # Поиск позиции
>>> where                           # Нашлась по смещению 4
4
>>> S = S[:where] + 'EGGS' + S[(where+4) :]
>>> S
'xxxxEGGSxxxxSPAMxxxx'
```

Метод **find** возвращает смещение, по которому обнаруживается подстрока (по умолчанию выполняя поиск с начала), или -1, если подстрока не найдена. Как упоминалось ранее, find представляет собой операцию поиска подстроки, подобную выражению in, но возвращает позицию найденной подстроки.

Еще один вариант предусматривает применение метода replace с третьим аргументом, чтобы ограничить его единственной подстановкой:

```
>>> S = ' xxxxSPAMxxxxSPAMxxxx'
>>> S.replace('SPAM' , 'EGGS')      # Заменить все
'xxxxEGGSxxxxEGGSxxxx'
>>> S.replace (' SPAM' , 'EGGS', 1) # Заменить одну
'xxxxEGGSxxxxSPAMxxxx'
```

Обратите внимание, что replace каждый раз возвращает новый строковый объект. Поскольку строки неизменяемы, метод никогда по-настоящему не модифицирует исходную строку на месте (поэтому результат необходимо присвоить переменной).

Тот факт, что операции конкатенации и метод replace каждый раз генерируют новые строковые объекты, на самом деле является потенциальным недостатком их использования для изменения строк. Если нужно применить много изменений к очень крупной строке, то производительность сценария можно улучшить, преобразовав строку в объект, который поддерживает изменения на месте:

```
>>> S = 'spammy'
>>> L = list(S)
>>> L
['s', 'p', 'a', 'm', 'm', 'y']
```

Встроенная функция **list** (вызов конструктора объекта) строит новый список из элементов любой последовательности — в данном случае извлекая символы строки и помещая их в список. Когда строка приобретает такую форму, в нее можно вносить множество изменений без генерации новой копии при каждом изменении:

```
>>> L[3] = 'x'              # Работает для списков, но не для строк
>>> L[4] = 'x'
>>> L
['s', 'p', 'a', 'x', ' x', 'y']
```

Если после внесения изменений список необходимо преобразовать обратно в строку (скажем, для записи в файл), тогда понадобится вызвать метод join:

```
>>> S = ' ' . join (L)
>>> S
'spaxxy'
```

На первый взгляд метод **join** может показаться слегка непонятным. Поскольку join представляет собой метод строк (не списков), он вызывается через заданный разделитель. Метод join объединяет вместе строки из списка (или другого итерируемого объекта) с разделителями между элементами списка; здесь в качестве разделителя используется пустая строка, чтобы преобразовать список в строку.

В более общем случае можно указывать любой строковый разделитель итерируемый объект со строками:

```
>>> 'SPAM'. join (['eggs' , 'sausage', 'ham', 'toast'])
'eggsSPAMsausageSPAMhamSPAMtoast‘
```

Поля данных здесь расположены по фиксированным смещениям и потому могут быть вырезаны из исходной строки. Такой прием подходит для разбора при условии, что компоненты данных имеют фиксированные позиции. Если взамен данные разделены посредством какого-нибудь разделителя, тогда извлечь компоненты можно путем разбиения. Такой прием будет работать, даже когда данные находятся в произвольных позициях внутри строки:

```
>>> line = 'aaa bbb ccc’
>>> cols = line.split()
>>> cols
[’aaa’, 'bbb', ’ccc’]
```

Строковый метод **split** разбивает строку на список подстрок согласно строке разделителя. В приведенном примере разделитель не указан, поэтому по умолчанию принимаются пробельные символы — строка разбивается по группам из одного или большего числа пробелов, табуляций и новых строк, а результатом будет список подстрок.

```
>>> line == ’bob, hacker, 40 ’
>>> line. split (' , ')
[’bob’, 'hacker’, ’40’]
```

Разделители могут быть длиннее одного символа:

```
>>> line = "i’mSPAMaSPAMlumberjack"
>>> line. split ("SPAM")
[”i’m“, 'a', ’lumberjack']
```

Другие строковые методы играют более специфичные роли — например, удаляют пробельные символы в конце строки текста, выполняют преобразование регистра символов, проверяют содержимое и выясняют наличие подстроки в конце или в начале:

```
>>> line = "The knights who say Ni!\n"
>>> line.rstrip()                    # удаляют пробельные символы
'The knights who say Ni!'
>>> line.upper()                     # преобразование регистра
'THE KNIGHTS WHO SAY NI! \n'
>>> line.isalpha()                   # проверка содержимого
False
>>> line.endswith( 'Ni! \n')         # наличие подстроки в конце
True
>>> line.startswith(’The')           # наличие подстроки в начале
True
```

Для достижения тех же результатов, которые получаются посредством строковых методов, иногда применяются альтернативные приемы; скажем, операцию членства in можно использовать для проверки наличия подстроки, а операции определения длины и нарезания — для имитации endswith \:

```
>>> line.find('Ni') ’ = -1        # Поиск через вызов метода или выражение
True
>>> 'Ni' in line                  # наличие подстроки
True
>>> sub = 'Ni!\n'
```

```
>>> line.endswith(sub)  # Проверка наличия подстроки в конце
True                    # через вызов метода или срез
>>> line[-len(sub) :] == sub
```

Задачи для самостоятельного выполнения

1.Обработка строк с помощью методов

1. Даны строки S и S0. Найти количество вхождений строки S0 в строку *S*.
2. Даны строки *S* и S_0. Удалить из строки *S* первую подстроку, совпадающую с S_0. Если совпадающих подстрок нет, то вывести строку *S* без изменений.
3. Даны строки *S*, S_1 и S_2. Заменить в строке *S* последнее вхождение строки S_1 на строку S_2.
4. Дан символ C и строка S. Удвоить каждое вхождение символа C в строку S.
5. Дан символ C и строки S, S0. Перед каждым вхождением символа C в строку S вставить строку S0.
6. Дано целое число N (> 0) и строка S. Преобразовать строку S в строку длины N следующим образом: если длина строки S больше N, то отбросить первые символы, если длина строки S меньше N, то в ее начало добавить символы «.» (точка).
7. Даны целые положительные числа N1 и N2 и строки S1 и S2. Получить из этих строк новую строку, содержащую первые N1 символов строки S1 и последние N2 символов строки S2

2. Анализ и преобразование слов в строке

8. Дана строка-предложение с избыточными пробелами между словами. Преобразовать ее так, чтобы между словами был ровно один пробел.
9. Дана строка, состоящая из русских слов, разделенных пробелами (одним или несколькими). Найти количество слов в строке.
10. Дана строка, состоящая из русских слов, набранных заглавными буквами и разделенных пробелами (одним или несколькими). Найти количество слов, которые содержат хотя бы одну букву «А».
11. Дана строка, состоящая из русских слов, набранных заглавными буквами и разделенных пробелами (одним или несколькими). Найти количество слов, которые начинаются и заканчиваются одной и тойже буквой.
12. Дана строка, состоящая из русских слов, разделенных пробелами (одним или несколькими). Найти длину самого короткого слова.
13. Дана строка, состоящая из русских слов, набранных заглавными буквами и разделенных пробелами (одним или несколькими). Преобразовать каждое слово в строке, заменив в нем все

последующие вхождения его первой буквы на символ «.» (точка). Например, слово «МИНИМУМ» надо преобразовать в «МИНИ.У.». Количество пробелов между словами не изменять.

14. Дана строка, состоящая из русских слов, набранных заглавными буквами и разделенных пробелами (одним или несколькими). Вывести строку, содержащую эти же слова, разделенные одним пробелом и расположенные в алфавитном порядке.

3. Другие задания на обработку строк

15. Дана строка, изображающая арифметическое выражение вида «<цифра>±<цифра>±…±<цифра>», где на месте знака операции «±» находится символ «+» или «–» (например, «4+7–2–8»). Вывести значение данного выражения (целое число).
16. Дан символ C, изображающий цифру или букву (латинскую или русскую). Если C изображает цифру, то вывести строку «digit», если латинскую букву — вывести строку «lat», если русскую — вывести строку «rus».
17. Дана строка. Если она представляет собой запись целого числа, то вывести 1, если вещественного (с дробной частью) — вывести 2; если строку нельзя преобразовать в число, то вывести 0. Считать, что дробная часть вещественного числа отделяется от его целой части десятичной точкой «.».
18. Дана строка-предложение на русском языке. Зашифровать ее, выполнив циклическую замену каждой буквы на следующую за нейв алфавите и сохранив при этом регистр букв («А» перейдет в «Б», «а» — в «б», «Б» — в «В», «я» — в «а» и т. д.). Букву «ё» в алфавитене учитывать («е» должна переходить в «ж»). Знаки препинания и пробелы не изменять.
19. Дана строка-предложение на русском языке и число K ($0 < K < 10$). Зашифровать строку, выполнив циклическую замену каждой буквына букву того же регистра, расположенную в алфавите на K-й позиции после шифруемой буквы (например, для $K = 2$ «А» перейдет в «В», «а» — в «в», «Б» — в «Г», «я» — в «б» и т. д.). Букву «ё» в алфавите не учитывать, знаки препинания и пробелы не изменять.
20. Дано зашифрованное предложение на русском языке (способ шифрования описан в предыдущем задании) и кодовое смещение K($0 < K < 10$). Расшифровать предложение.
21. Дана строка, содержащая *полное имя файла*, то есть имя диска, список каталогов (путь), собственно имя и расширение. Выделить из этой строки имя файла (без расширения).
22. Дана строка, содержащая *полное имя файла*, то есть имя диска, список каталогов (путь), собственно имя и расширение. Выделить из этой строки расширение файла (без предшествующей точки).
23. Дана строка, содержащая латинские буквы и скобки трех видов: «()», «[]», «{}». Если скобки расставлены правильно (то есть каждой

открывающей соответствует закрывающая скобка того же вида), то вывести число 0. В противном случае вывести или номер позиции, в которой расположена первая ошибочная скобка, или, если закрывающих скобок не хватает, число –1.

Дополнительные задания по теме

1. Дан символ C и строки S, S0. Перед каждым вхождением символа C в строку S вставить строку S0.
2. Даны строки S и S0. Проверить, содержится ли строка S0 в строке S. Если содержится, то вывести True, если не содержится, то вывести False.
3. Даны строки S и S0. Удалить из строки S последнюю подстроку, совпадающую с S0. Если совпадающих подстрок нет, то вывести строку S без изменений.
4. Дана строка, содержащая по крайней мере один символ пробела. Вывести подстроку, расположенную между первым и вторым пробелом исходной строки. Если строка содержит только один пробел, то вывести пустую строку.
5. Дана строка, содержащая по крайней мере один символ пробела. Вывести подстроку, расположенную между первым и последним пробелом исходной строки. Если строка содержит только один пробел, то вывести пустую строку.
6. Дана строка, состоящая из русских слов, разделенных пробелами (одним или несколькими). Найти длину самого длинного слова.
7. Дана строка-предложение на русском языке. Подсчитать количество содержащихся в строке знаков препинания.
8. Дана строка, содержащая полное имя файла. Выделить из этойстроки название первого каталога (без символов «\»). Если файл содержится в корневом каталоге, то вывести символ «\».
9. Дана строка-предложение на русском языке. Преобразовать строку так, чтобы каждое слово начиналось с заглавной буквы. Словом, считать набор символов, не содержащий пробелов и ограниченный пробелами или началом/концом строки. Слова, не начинающиеся с буквы, не изменять.
10. Дана строка-предложение на русском языке. Подсчитать количество содержащихся в строке гласных букв.
11. Дана строка, содержащая цифры и строчные латинские буквы. Если буквы в строке упорядочены по алфавиту, то вывести 0; в противном случае вывести номер первого символа строки, нарушающего алфавитный порядок.
12. Дана строка, содержащая латинские буквы и круглые скобки. Если скобки расставлены правильно (то есть каждой открывающей соответствует одна закрывающая), то вывести число 0. В противном случае вывести или номер позиции, в которой расположена первая

ошибочная закрывающая скобка, или, если закрывающих скобок не хватает, число –1.

Контрольные вопросы по теме «Строки»

1. К какому типу относятся строки (последовательности / непоследовательности, изменяемые / неизменяемые) и, что это означает?
2. Как осуществляется доступ к отдельному элементу строки?
3. Как получить срез строки?
4. Что означает термин неизменяемый объект, строки являются изменяемыми или нет?
5. Как называется кодировка, поддерживающая кодирование буквенноцифровых символов? Расскажите о ее структуре.
6. Перечислите основные функции для работы с символами. Приведите примеры.
7. Перечислите методы работы со строками, позволяющие преобразовывать символы строки к различным регистрам клавиатуры.
8. Какой метод позволяет разбить строку на подстроки? Напишите его синтаксис.
9. Какой метод отвечает за преобразование строки в список? Напишите его синтаксис.
10. Какой метод отвечает за обратное преобразование списка в строку? Напишите его синтаксис.
11. Приведите примеры базовых алгоритмов строк.
12. Какими способами можно получить срез строки?